

Docker — Cheat Sheet

Disposable environments for OSINT: run, inspect, destroy. Compose and Dockerfile basics.

Three concepts

Term	What it is
Image	a read-only template (a filesystem + metadata)
Container	a running (or stopped) instance of an image
Volume	persistent storage that outlives a container

The container is disposable; the **volume** (or a bind mount) is the only thing that survives. That volatility is an OPSEC feature: `docker rm` and the workspace is gone.

Run & manage

```
docker run hello-world                # pull + run a test image
docker run -it --rm debian:bookworm-slim bash # throwaway shell; exit destroys it
docker run -d --name web -p 8088:80 nginx:alpine # detached web server, port 8088->80
docker ps                             ; docker ps -a      # running / all containers
docker exec -it web sh                 # shell into a running container
docker logs -f web                     # follow its output
docker stop web                       ; docker rm web     # stop then remove
docker rm -f web                       # force-remove a running container
```

Images

```
docker images                        # list local images
docker pull nginx:alpine            # fetch an image
docker build -t myimg:latest -f Dockerfile . # build from a Dockerfile
docker rmi myimg:latest              # remove an image
docker save myimg:latest | gzip > myimg.tgz # export (USB transfer)
docker load -i myimg.tgz             # import
```

Volumes & bind mounts

```
docker volume create data            # named volume
docker run -v data:/var/lib/app myimg # mount a named volume
docker run -v "$PWD/work:/work" myimg # bind-mount a host folder (evidence bridge)
docker run -v "$PWD/work:/work:ro" myimg # read-only mount
```

Ports & networking

```
docker run -p 5678:5678 n8nio/n8n    # host:container port
docker network ls                     # networks
docker run --network host myimg       # share host network (Linux)
docker run --cap-drop ALL --security-opt no-new-privileges myimg # least privilege
```

docker compose

```
docker compose up -d                # start all services (detached)
docker compose up -d n8n            # start one service
docker compose ps                   ; docker compose logs -f
docker compose down                 # stop + remove containers (keep volumes)
docker compose down -v              # ALSO delete volumes: nothing survives
```

```
# docker-compose.yml
services:
  n8n:
    image: n8nio/n8n:latest
    ports: ["5678:5678"]
    environment: [GENERIC_TIMEZONE=UTC, N8N_DIAGNOSTICS_ENABLED=false]
    volumes: ["n8n_data:/home/node/.n8n"]
volumes:
  n8n_data:
```

Dockerfile basics

```
FROM debian:bookworm-slim
ENV TZ=UTC LANG=C.UTF-8                # no regional leak
RUN apt-get update && apt-get install -y curl exiftool && rm -rf /var/lib/apt/lists/*
RUN useradd -m analyst                 # non-root
USER analyst
WORKDIR /workspace
ENTRYPOINT ["bash"]
```

Cleanup

```
docker ps -a          ; docker images      # see what exists
docker system df      # disk used by Docker
docker container prune ; docker image prune # remove stopped / dangling
docker system prune -a --volumes          # nuke everything unused (careful)
```

OSINT / OPSEC usage

- One disposable container per case; `--rm` so nothing persists but the mounted workspace.
- Run tools without installing them on the host (`docker run --rm ... tool`).
- Neutral hostname, `TZ=UTC` , non-root user, `--cap-drop ALL` (Day 1 SecOps container).
- Secrets via env vars / compose secrets, never baked into the image.

Golden rule: image = your clean snapshot, container = the instance, volume = what you keep. Tear down with `down` `-v` when the case is closed.